

CTF
HACKTHEBOX
X YNOV CAMPUS

Capture the Flag - Édition 2026

L'ÉVÉNEMENT ET L'EXPERTISE

Une immersion totale dans le hacking éthique, encadrée par des experts mondiaux
et une compétition féroce.

Encadrement & Performance



Experts HackTheBox

Venue exceptionnelle de deux ingénieurs cybersécurité HTB, venus directement de **Londres** et de **Grèce**.



Équipe & Distinction

Compétition en escouades de **5 membres**.

Grâce à la résolution d'un maximum de challenges critiques, j'ai eu l'honneur de finir **MVP** de mon équipe en validant un maximum de flags.



Ground Station Breach

Objectif du Challenge

Intercepter des images satellites classifiées relayées par un fournisseur SLE ennemi.

 Connexion au flux **RAF**

 Capture des trames **CCSDS**

 Reconstruction via script Python

```
SAT-IMG-2024-001  
ALT: 450KM  
LAT/LON: CLASSIFIED  
  
HTB{sp4c3_l1nk_fr4m3s_fr0m_0rb1t}
```

Reconnaissance & Analyse



Analyse Réseau

`tcpdump`

Découverte du peer_identifier :
`GROUND_STATION_ALPHA.`



Fuzzing Web

`gobuster`

Identification de l'API sur le port 30478.



Identification

`/api/service-instances/`

Cible : `EARTH_OBS_CLASSIFIED.`

Élévation de Privilèges

Le Hack API (PATCH Method)

Utilisation de la méthode HTTP **PATCH** pour modifier la configuration de la ressource à distance.

```
curl -X PATCH -d '{"classification":  
"UNCLASSIFIED"}'
```

Code 204 reçu : Injection réussie !

Capture & Reconstruction

```
(dirty@kali)-[~/ctf/HTBxYC/crypto_repeated_maleficence]
$ cat solve.py
# solve.py

# — Étape 1 : Charger le texte chiffré depuis l'hexadécimal —
with open('encrypted.txt', 'r') as f:
    hex_data = f.read().strip()

ciphertext = bytes.fromhex(hex_data)

# — Étape 2 : Attaque par texte clair connu (KPA) —
# On sait que le flag commence par "HTB{"
known_plaintext = b"HTB{"
key = bytearray(5)

# On retrouve les 4 premiers octets de la clé : K = C XOR P
for i in range(4):
    key[i] = ciphertext[i] ^ known_plaintext[i]

print(f"[*] 4 premiers octets de la clé trouvés : {key[:4].hex()}")
print("[*] Brute-force du 5ème octet en cours...")

# — Étape 3 : Brute-force du dernier octet —
for k4 in range(256):
    key[4] = k4

    # Déchiffrement avec la clé testée
    decrypted = bytearray()
    for i in range(len(ciphertext)):
        decrypted.append(ciphertext[i] ^ key[i % 5])

    # Vérification : on garde le résultat si c'est de l'ASCII imprimable et que ça finit par "
    try:
        flag = decrypted.decode('ascii')
        if flag.endswith('}') and all(32 ≤ ord(c) ≤ 126 for c in flag):
            print(f"\n[+] Clé complète trouvée : {key.hex()}")
            print(f"🚩 FLAG : {flag}")
            break
    except UnicodeDecodeError:
        # Si ça ne se décode pas en ASCII proprement, c'est que ce n'est pas la bonne clé
        continue

(dirty@kali)-[~/ctf/HTBxYC/crypto_repeated_maleficence]
$ python3 solve.py
[*] 4 premiers octets de la clé trouvés : 3a580378
[*] Brute-force du 5ème octet en cours...

[+] Clé complète trouvée : 3a580378e0
🚩 FLAG : HTB{h0r_1C_w34{_w15x_kn0gn_p1$1n53h5}
```

Finalisation Opérationnelle

Développement d'un script Python exploitant la bibliothèque [sle](#).

Interception du flux binaire via les logs DEBUG.

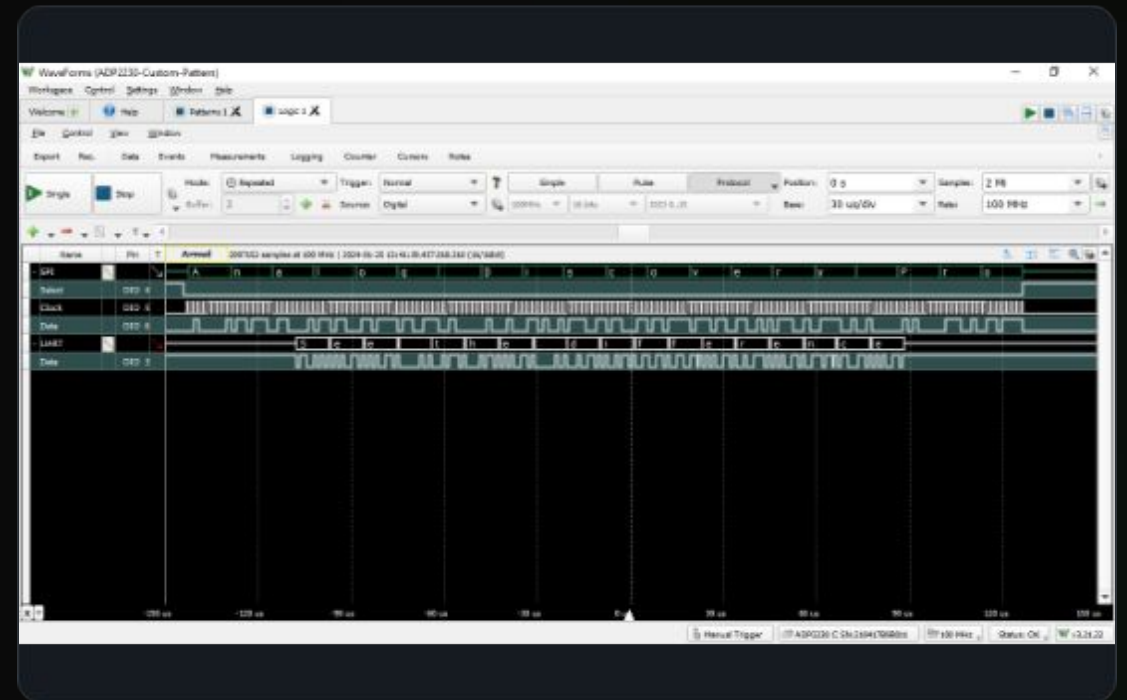
Reconstruction de l'image avec [Pillow](#) pour récupérer le flag.

Hidden Logs

Analyse du Signal UART

Analyse de la suite binaire (0/1) d'une capture logique (Raw Logic Capture) de l'appareil.

Déduction du format UART : 1 Start bit (0), 8 Data bits (LSB first), 1 Stop bit (1).





Forensic : SpyBackup

Catégorie	Valeur / Réponse	Méthodologie
Endpoint Exfil	<code>spybackup.htb/upload</code>	KPA sur libspybackup.so
Clé XOR (4-byte)	<code>0x42, 0x13, 0x37, 0x99</code>	Calcul inverse XOR
Timestamp Exfil	<code>2026-01-24T10:57:27Z</code>	Analyse d'ID SQLite (data.db)



Challenge Cryptographie : RSA Factoring

Attaque sur une vulnérabilité d'implémentation RSA ($n = p^r$). Calcul de $\phi(n)$ et de l'exposant privé d pour décrypter le flag.

```
(venv)-(dirty@kali)-[~/ctf/HTBxYC/crypto_rsactftool]
$ python3 solve.py
Chargement de key.pem ...
Chargement de flag.enc ...
Recherche de la racine entière ...
[+] Trouvé ! p = 710360121478080504193025638482836808691249126334909608653047579435
83941629952501710284455656629407157063470845309072204991
[+] Puissance r = 19
[+] Exposant privé d trouvé.

🚀 Flag récupéré avec succès : HTB{s1z3_d03$n5_m4553r____i-5h_r00t_1$_w45ch1ng}
```



 JWT Debugger

Debugger

Introduction

Libraries

Ask



JWT Decoder

JWT Encoder

Paste a JWT below that you'd like to decode, validate, and verify.

Generate example

Select signing algorithm

Encoded Token

☐ Enable auto-focus

>_ JSON Web Token (JWT)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwidXNlcl9yb2xlIjoiYWRTaW4iLCJmbGFuIjoiSFRCe0YxWF9ZMFVyX3g1NV9NNG5fMXQ1X2Q0bmdlcjB1cyFfOWIwYTc1ODZkOGI3ODc2ZDM1NGRhMjY0YTdlODMSY2Z9IiwiaWF0IjoxNzgyODI0OTMwMwQ.SKuiChjLWIEuFYw-8M9EXns7CuRqknpfXZDuWCPWmka
```

Decoded Header

JSON

Claims Breakdown

```
{  "alg": "HS256",  "typ": "JWT"}
```

Decoded Payload

JSON

Claims Breakdown

```
{  "username": "admin",  "user_role": "admin",  "flag": "HTB{F1X_Y0Ur_x55_M4n_1t5_d4nger0us!_9b0a7586d8b7876d354da264a7e839cf}",  "iat": 1782824930}
```



Classement & Scoreboard Final

Performance de l'équipe : Top 10 atteint grâce à une solide cohésion et une validation continue des flags

CTF Ended×



Rank 9

GamblingLovers
CTF Team

21/34 Flags Pwned	France Team Country	Durrt Team MVP
-----------------------------	-------------------------------	--------------------------

CTF Bordeaux Ynov Campus ...
CTF Name

[View Scoreboard](#)